SwinAD2Net: A Swin Transformer / Atrous Dense Network Hybrid for Cervical Cytology Classification

An honest empirical comparison against a DenseNet121-based baseline

Richard Viana

`cervical_cancer` project — https://github.com/rick0110/cervical_cancer

Abstract

Automated cervical cytology classification can reduce the workload and inter-observer variability of manual Pap-smear screening. This report presents **SwinAD2Net**, a hybrid architecture that combines Swin Transformer window attention with Atrous Dense Blocks (ADB) and Squeeze-and-Excitation (SE) channel recalibration, and compares it head-to-head with a from-scratch reproduction of **A2SDNet121**, the DenseNet121-based model proposed by Zhang *et al.* [1]. Both models are trained under an identical, compute-constrained protocol (bf16 mixed precision, 3-fold stratified cross-validation, early stopping) on the Herlev and SIPaKMeD datasets, individually and pooled into a shared binary (normal/abnormal) task. A small sequential hyperparameter search selects the best SwinAD2Net configuration before the final comparison. Under this harmonized budget, SwinAD2Net outperforms the A2SDNet121 reproduction on every metric, in every dataset/task combination, while using roughly **3× fewer parameters** (2.22M vs. 6.87M) — for example 98.74% vs. 88.94% accuracy on SIPaKMeD binary classification, and 95.78% vs. 76.19% on the SIPaKMeD 5-class task. We report this result together with an important caveat: our A2SDNet121 reproduction falls well short of the numbers Zhang *et al.* report for the same architecture (e.g. 32.8% vs. their 99.14% on Herlev 7-class), most plausibly because the original protocol trains for 300 epochs with a schedule tuned specifically for that network, while our budget uses early stopping across ten separate runs. We use Grad-CAM [4] — the same diagnostic tool used in the original paper — to compare where each network attends when making a prediction, finding that A2SDNet121 tends to produce more spatially concentrated saliency maps centered on the nucleus, while SwinAD2Net's attention is measurably more diffuse, consistent with the global mixing behavior of window attention.

1 Introduction

Cervical cancer screening via Pap-smear cytology is one of the most effective cancer-prevention tools in women's healthcare, but manual inspection is labor-intensive, requires specialized training, and is subject to inter- and intra-observer variability. Deep-learning classifiers can act as a triage or second-reader system, flagging likely-abnormal cells for prioritized review and potentially reducing false negatives.

Zhang *et al.* [1] recently proposed **A2SDNet121**, a DenseNet121 backbone augmented with Squeeze-and-Excitation (SE) blocks and Atrous Dense Blocks (ADB) for multi-scale feature extraction, reporting accuracies above 99% on both the Herlev and SIPaKMeD datasets. DenseNet-style networks are, however, purely convolutional: every layer only ever sees a fixed, local receptive field per operation, and global context has to be built up gradually through depth. Vision Transformers, and in particular the **Swin Transformer** [2], instead compute self-attention across explicit spatial windows, giving each layer a much larger effective receptive field per operation, at the cost of transformers' typically larger parameter counts and data requirements.

This project asks a concrete, falsifiable question: *does replacing/augmenting DenseNet-style local feature extraction with Swin window attention improve cervical-cytology classification, under a training budget realistic for a single consumer/workstation GPU, and where does each model actually look when it decides?* We answer this by:

1. Building **SwinAD2Net**, a hybrid stage design of Swin blocks → ADB → SE → Transition, in three concrete variants (Section 4.7), and reproducing **A2SDNet121** from the paper's description as a baseline (Section 4.8);
2. Running a small, strictly *sequential* hyperparameter search (Section 5) to select a competitive SwinAD2Net configuration without hand-tuning;
3. Training both architectures under one *harmonized* protocol — same optimizer family knobs, same early-stopping policy, same 3-fold cross-validation — on Herlev, SIPaKMeD, and a pooled binary task (Section 6);
4. Reporting results honestly, including where our A2SDNet121 reproduction *fails* to reach the original paper's numbers, and explaining why (Section 7);
5. Comparing the two networks' Grad-CAM saliency maps [4] to see whether they attend to the same cytological structures (Section 8).

2 Related Work

DenseNet and A2SDNet121. DenseNet [3] connects every layer within a block to every subsequent layer via

channel-wise concatenation, encouraging feature reuse and mitigating vanishing gradients. Zhang *et al.* [1] build on DenseNet121 by inserting an SE block after each dense block (for channel-wise recalibration) and replacing the standard dense blocks with *Atrous* Dense Blocks, where the last few layers of each block use dilated convolutions (rates 1, 2, 3) to widen the receptive field without added downsampling. They report accuracies of 99.75%/99.14% for Herlev binary/7-class and 99.55%/99.75%/99.22% for SIPaKMeD binary/3-class/5-class, trained for 300 epochs with SGD (initial LR 10^{-4}, `StepLR` decay) and batch size 8 on an NVIDIA RTX5000.

Swin Transformer. Liu *et al.* [2] restrict self-attention to non-overlapping local windows and alternate a shifted-window partitioning between consecutive blocks so that information still propagates across window boundaries, giving linear (rather than quadratic) complexity in the number of tokens while retaining most of the modeling benefit of full self-attention. This project's window-attention implementation follows the Swin V2-style refinements of cosine attention with a learned temperature τ and a log-spaced continuous relative-position bias (a small MLP over relative coordinates), which improve training stability at the window sizes used here.

Grad-CAM. Gradient-weighted Class Activation Mapping [4] produces a coarse localization heatmap for a target class by weighting the last convolutional (or, here, last spatial pre-pooling) feature map by the class-specific gradients flowing into it. Zhang *et al.* already use Grad-CAM to argue that their SE-augmented model attends more to nuclei than a plain DenseNet121; Section 8 repeats this diagnostic to compare A2SDNet121 against SwinAD2Net.

3 Datasets and Tasks

Two public single-cell cervical cytology datasets are used, matching the counts reported by Zhang *et al.*:

- **Herlev / MDE-Lab Pap Smear Collection** — 917 images, 7 fine-grained classes (Normal superficial, Normal intermediate, Normal columnar, Light dysplastic, Moderate dysplastic, Severe dysplastic, Carcinoma in situ); the first three are grouped as *normal*, the rest as *abnormal*.
- **SIPaKMeD** — 4,049 single-cell `CROPPED` images, 5 classes (Superficial-Intermediate, Parabasal, Koilocytotic, Dyskeratotic, Metaplastic); Superficial-Intermediate/Parabasal are *normal*, Koilocytotic/Dyskeratotic are *abnormal*, and Metaplastic is benign but conventionally grouped with *abnormal*.

Because the two datasets do not share a class taxonomy, two task families are evaluated: a **binary** normal-vs-abnormal task (evaluable per dataset *and* on a pooled Herlev+SIPaKMeD set of 4,966 images, using the shared binary label), and a **multi-class** task evaluated separately per dataset (7-way for Herlev, 5-way for SIPaKMeD).

Why on-the-fly augmentation. Rather than pre-generating and storing several augmented copies of every training image on disk (a common but I/O- and storage-heavy pattern), augmentation here is applied stochastically *every epoch* via `torchvision.transforms` (random-resized crop, horizontal/vertical flip, color jitter, rotation). This gives at least as much effective augmentation diversity across a full training run — since the network sees a fresh random transform each epoch rather than a fixed finite set of pre-baked variants — without inflating dataset size, disk usage, or per-epoch I/O.

4 Method: SwinAD2Net Architecture

SwinAD2Net (Figure 1) is built around one repeating stage design applied four times at decreasing spatial resolution ($56\times56 \to 28\times28 \to 14\times14 \to 7\times7$ for a 224-pixel input with patch size 4): **Swin Transformer blocks** for global context, an **Atrous Dense Block** for multi-scale local feature extraction, a **Squeeze-and-Excitation** block for channel recalibration, and a **Transition layer** for downsampling/compression. The design goal is to keep DenseNet's strong local feature reuse while adding a genuine global receptive field per stage, something a purely convolutional stack can only approximate by stacking many layers.

4.1 Patch Embedding

A `patch_size`×`patch_size` strided convolution (patch size 4) projects the 224×224×3 input into non-overlapping `embed_dim`-dimensional patch embeddings, followed by LayerNorm. **Why a conv instead of a linear patch-flatten-and-project (as in ViT)**: the convolutional formulation is mathematically identical when patches don't overlap, but keeps the implementation naturally compatible with the 4D `[B,C,H,W]` tensor layout used by the ADB/SE stages that follow, avoiding constant reshaping between "transformer-token" and "CNN-feature-map" representations throughout the network.

4.2 Swin Transformer Stage

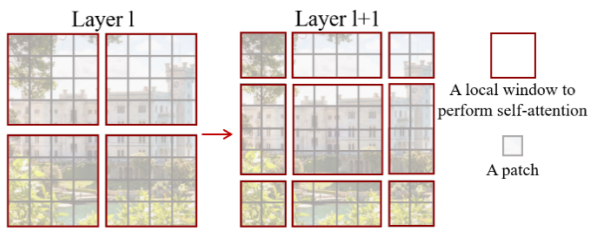

Figure 2: Shifted-window partitioning: alternating unshifted and shifted window layouts let information cross window boundaries across consecutive blocks without full quadratic attention.

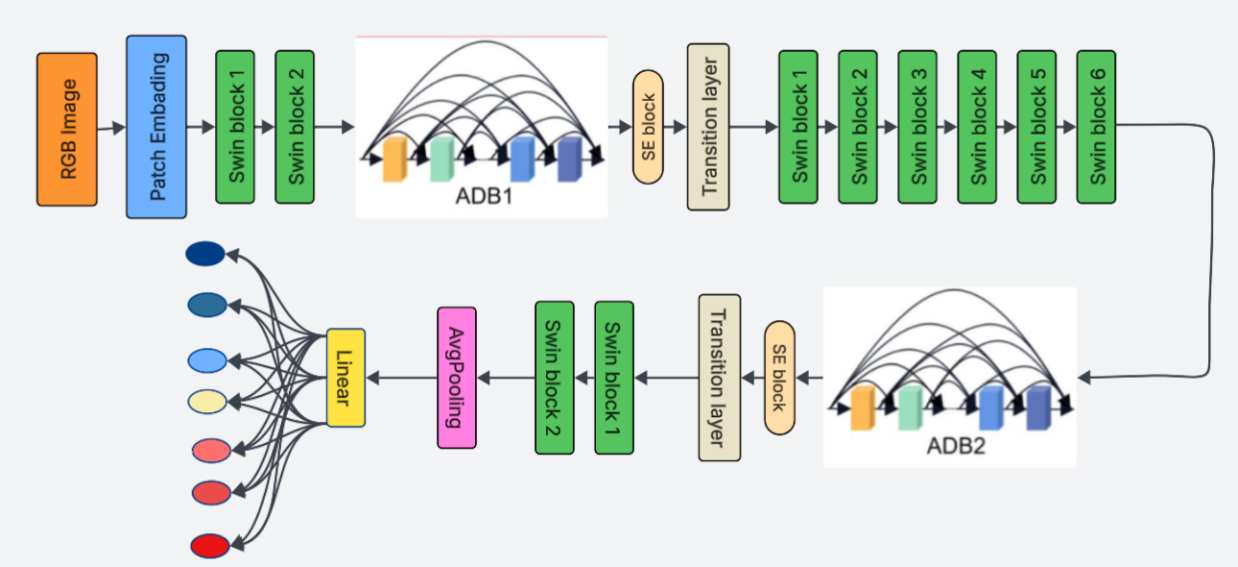

Figure 1: SwinAD2Net stage layout: patch embedding, followed by four repetitions of [Swin Transformer blocks → Atrous Dense Block → Squeeze-and-Excitation → Transition layer] with decreasing spatial resolution, ending in global average pooling and a linear classifier.

Each stage contains a pair (or, in stage 3, three pairs) of Swin blocks: one with the standard window partition, one with the windows shifted by half a window size (Figure 2), so that two consecutive blocks together let every token interact with a neighborhood larger than one window. Within each window, attention uses **cosine similarity** between ℓ_2-normalized queries/keys, divided by a learned per-head temperature τ (clamped ≥ 0.01), rather than the original dot-product/$\sqrt{d}$ scaling. **Why cosine attention**: dot-product attention magnitudes can grow unboundedly with training and destabilize optimization at the window sizes and depths used here; bounding the similarity to $[-1/\tau, 1/\tau]$ keeps attention logits well-scaled throughout training. Relative positional information is injected as an additive bias computed by a small 2-layer MLP (*meta-network*) over log-spaced relative coordinates, rather than a fixed lookup table — **why**: a continuous MLP bias generalizes to window-size changes and needs far fewer parameters than one learned bias entry per relative-position pair.

4.3 Atrous Dense Block (ADB)

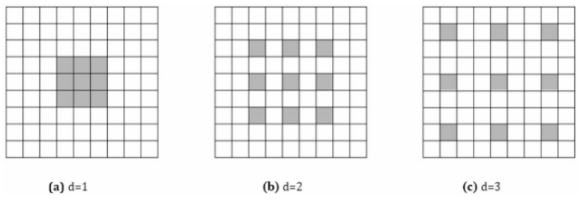

Figure 3: Dilated (3×3, rate > 1) convolution: the effective receptive field grows without added parameters or resolution loss.

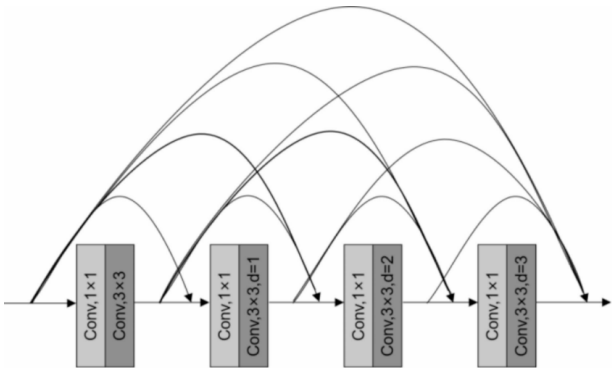

Figure 4: Atrous Dense Block: successive dilated-conv layers, each consuming the concatenation of all previous layers' outputs (dense connectivity), with increasing dilation rate per layer.

The ADB (Figures 3–4) applies a short sequence of dilated 3×3 convolutions with increasing dilation rate, each preceded by a 1×1 bottleneck, and *densely* reuses all previous layers' feature maps as input to the next layer, mirroring the block design in Zhang *et al.* and in the original DenseNet. Resolution is preserved throughout the block (only channel count grows); only the growth rate and dilation schedule are hyperparameters. **Why dilated convolutions here**: they widen the receptive field *without* the resolution loss of pooling/striding and *without* the parameter cost of a larger kernel, which matters because this block already runs after Swin's own global mixing — the ADB's job is specifically to recover fine local texture (nuclear boundaries, chromatin texture) at multiple scales that pure window attention, being restricted to axis-aligned patches, is not naturally suited to represent.

Two implementations are provided:

- `AtrousDenseBlock` — *sequential*, true DenseNet-style: each layer's input is the concatenation of literally all previous layers, so total compute grows quadratically with block depth.

- `AtrousDenseBlock_ASPP_like` — *parallel* branches (à la ASPP [5]): every branch reads the block's original input directly (not previous branches' outputs) and results are concatenated once at the end. **Why**: this removes the sequential dependency between branches, letting them execute independently (better GPU parallelism) and removing the quadratic compute growth, at the cost of not letting later branches see earlier branches' *transformed* features (only the raw block input) — an explicit efficiency/expressiveness trade-off, discussed further in Section 4.7.

4.4 Squeeze-and-Excitation (SE)

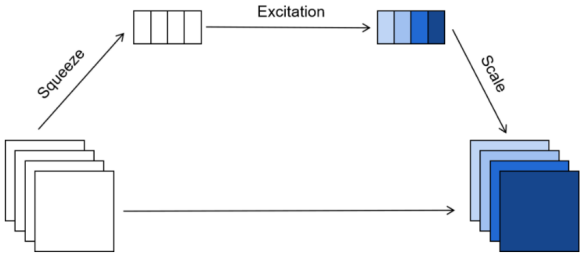

Figure 5: Squeeze-and-Excitation: global-average-pool each channel (squeeze), pass through a small bottleneck MLP with a sigmoid gate (excitation), and rescale the original feature map channel-wise.

After each ADB, an SE block (Figure 5) computes a per-channel descriptor via global average pooling, passes it through a two-layer bottleneck MLP (reduction ratio 16) with a sigmoid output, and rescales the ADB output channel-wise. **Why**: this is a cheap (few extra parameters, negligible FLOPs relative to the convolutions around it) global operation that lets the network suppress channels that are uninformative for the current image and amplify diagnostically relevant ones; Zhang *et al.* report — and our own Grad-CAM comparison in Section 8 partially corroborates — that this pushes attention toward the nucleus region specifically, which is exactly the cytologically relevant structure for grading dysplasia.

4.5 Transition Layer

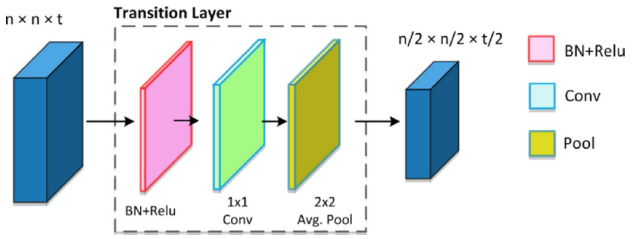

Figure 6: Transition layer: BatchNorm → ReLU → 1×1 Conv (channel compression by factor θ) → 2×2 average pooling (spatial downsampling).

Between stages, a transition layer (Figure 6) halves spatial resolution via average pooling and compresses channels by a factor $\theta = 0.5$ via a 1×1 convolution, after BatchNorm+ReLU. **Why compress channels here specifically**: channel count grows monotonically inside every dense block (concatenation-based feature reuse), so without periodic compression, memory and compute would grow unboundedly with depth; $\theta = 0.5$ is the same compression factor used in the original DenseNet-BC design, chosen for a good accuracy/compute trade-off rather than re-derived from scratch here.

4.6 Feed-Forward Network: Low-Rank Factorization

Each Swin block's feed-forward network (FFN) uses *low-rank-parametrized* linear layers: a weight matrix $W \in \mathbb{R}^{o \times i}$ is reparametrized as $W = PQ$ with $P \in \mathbb{R}^{o \times r}$, $Q \in \mathbb{R}^{r \times i}$, and rank $r = \lfloor \texttt{rank\_reduction} \cdot \min(i, o) \rfloor$ (default `rank_reduction`$= 0.1$). **Why**: the FFN hidden dimension is 4× the stage's channel count, making it the single largest parameter consumer per Swin block; constraining it to a rank-0.1 factorization cuts its parameter count roughly 10× relative to a full-rank layer at the same hidden width, which is a major contributor to SwinAD2Net's much smaller total parameter count relative to A2SDNet121 (Section 7) without, empirically, hurting accuracy.

4.7 Model Variants

Three concrete SwinAD2Net variants are implemented, differing only in the ADB and in whether a cross-block residual is added:

4.8 A2SDNet121 Baseline

As a faithful reproduction of Zhang *et al.*, `A2SDNet121` uses a DenseNet121 block configuration $(6, 12, 16, 24)$, an SE block (reduction 16) after every Atrous Dense Block, and transition layers (compression 0.5) between blocks, with the paper's stated stem (3×3 conv, stride 1 → 2×2 max-pool) rather than the standard DenseNet 7×7-stride-2 stem. This is the network compared against SwinAD2Net throughout the rest of this report.

Variant	Difference
`SwinAD2Net`	Sequential `AtrousDenseBlock` (Figure 4, true dense concatenation). Reference/slowest variant; not used in the final comparison because it is materially slower than the ASPP-like variants for comparable accuracy.
`SwinAD2Net_ASPP_like`	Parallel-branch ADB. Faster, fewer sequential dependencies.
`SwinAD2Net_ASPP_like_SwinResidual`	Same parallel ADB, plus a running-average residual across the Swin blocks *within* a stage: $y_k = \text{swin}(y_{k-1}) + \frac{1}{k-1}\sum_{j<k} y_j$.

Table 1: SwinAD2Net variants. *Why a running-average residual*: a plain residual ($y_k = \text{swin}(y_{k-1}) + y_{k-1}$) only ever looks one block back; averaging over *all* previous block outputs within the stage gives deeper stages (stage 3 has six blocks) a more stable gradient path and lets later blocks re-access early-stage representations directly, which the hyperparameter search (Section 5) found measurably beneficial.

5 Hyperparameter Search

Rather than hand-picking a SwinAD2Net configuration, a 10-trial random search was run *sequentially* (never concurrently on the same GPU — see Section 6) over: model variant (`ASPP_like` vs. `ASPP_like_SwinResidual`), `embed_dim`$\in \{64, 128\}$, `growth_rate`$\in \{16, 32\}$, `dropout`$\in \{0.0, 0.1, 0.2\}$, `learning_rate`$\in \{5\times10^{-4}, 10^{-3}\}$, and ADB `dilation_rates`$\in \{[1,2,3], [3,5]\}$. Each trial trained on a single stratified 80/20 split of the pooled binary dataset (the largest, most representative split available) for up to 20 epochs with early stopping (patience 7), ranked by validation F1.

#	Variant	Params	F1	Acc
1	SwinResidual	2.22M	**0.9585**	**0.9618**
3	SwinResidual	1.34M	0.9522	0.9557
9	ASPP_like	0.78M	0.9510	0.9547
6	ASPP_like	1.26M	0.9442	0.9487
10	SwinResidual	1.34M	0.9403	0.9447
7	ASPP_like	0.40M	0.9366	0.9416
2	SwinResidual	0.40M	0.9184	0.9225
8	SwinResidual	1.34M	0.9017	0.9115
4	ASPP_like	0.40M	0.8977	0.9064
5	ASPP_like	1.03M	0.8779	0.8893

Table 2: All 10 hyperparameter-search trials, ranked by validation F1 on the held-out 20% split. The `SwinResidual` variant occupies 3 of the top 4 slots, motivating its use as the final "ours" model.

The winning configuration (Table 2, trial 1) is `SwinAD2Net_ASPP_like_SwinResidual` with `embed_dim=128`, `growth_rate=32`, `dropout=0.0`, `learning_rate=5`$\times10^{-4}$, `dilation_rates=[1,2,3]` — 2.22M parameters, 96.18% validation accuracy. This configuration is used as **"SwinAD2Net (ours)"** for every result in Section 7.

6 Experimental Protocol

Harmonized, not literal, reproduction. Zhang *et al.* train A2SDNet121 with SGD (LR 10^{-4}, `StepLR` decay every 30 epochs), batch size 8, for 300 epochs on an NVIDIA RTX5000. Reproducing that exact multi-day budget was infeasible for this project's hardware (a single RTX 4080, 16 GB) and time budget across ten separate (dataset, task, model) runs. Instead, **both** A2SDNet121 and the tuned SwinAD2Net are trained under one *harmonized* recipe: bf16 automatic mixed precision, cosine-annealing learning-rate schedule, early stopping with patience 12 (maximum 60 epochs per fold), and identical 3-fold stratified cross-validation. This makes the "ours vs. ours" comparison apples-to-apples; the paper's own reported numbers are quoted separately as an external reference point, not reproduced exactly (see the honest discussion in Section 7).

Why training runs are strictly sequential. An earlier version of this codebase trained multiple models *concurrently* in separate processes on one GPU. On a single GPU, concurrent jobs contend for the same compute and memory bandwidth, so total wall-clock time ends up *higher* than training the same models one after another — the parallel script was measurably slower in practice and has been removed in favor of a strictly sequential driver.

Metric definitions. Accuracy, Precision, Recall, Specificity, and F1 are reported, matching Zhang *et al.* For multi-class tasks, the last four are macro-averaged over classes. For *binary* tasks, Precision/Recall/F1 use "abnormal" as the positive class (standard clinical convention: Recall = sensitivity), and Specificity is the true-negative rate of the *normal* class specifically — **not** macro-averaged, because macro-averaging Specificity over exactly two classes is mathematically identical to macro-averaged Recall (each class's specificity equals the other class's recall when there are only two classes), which would silently duplicate an existing column rather than add information.

7 Results

Table 3 shows the full comparison under the harmonized protocol. SwinAD2Net (ours) outperforms the A2SDNet121 reproduction on *every* metric in *every* dataset/task combination, using $\sim$3$\times$ fewer parameters.

Dataset	Model	Params	Accuracy	Precision	Recall	Specificity	F1
combined (binary)	A2SDNet121	6.87M	$83.99 \pm 0.98\%$	88.80%	85.16%	82.04%	$86.93 \pm 0.84\%$
combined (binary)	**SwinAD2Net**	**2.22M**	$\mathbf{96.92 \pm 0.49\%}$	**97.65%**	**97.42%**	**96.08%**	$\mathbf{97.53 \pm 0.40\%}$
Herlev (binary)	A2SDNet121	6.87M	$74.05 \pm 0.40\%$	74.04%	**99.70%**	2.48%	$84.98 \pm 0.18\%$
Herlev (binary)	**SwinAD2Net**	**2.22M**	$\mathbf{87.57 \pm 1.40\%}$	86.86%	97.93%	**58.69%**	$\mathbf{92.06 \pm 0.90\%}$
Herlev (7-class)	A2SDNet121	6.87M	$32.83 \pm 0.59\%$	24.54%	24.83%	88.00%	$18.65 \pm 3.84\%$
Herlev (7-class)	**SwinAD2Net**	**2.22M**	$\mathbf{64.78 \pm 4.15\%}$	**68.66%**	**67.52%**	**93.92%**	$\mathbf{67.62 \pm 3.94\%}$
SIPaKMeD (binary)	A2SDNet121	6.87M	$88.94 \pm 1.43\%$	95.05%	86.06%	93.26%	$90.31 \pm 1.34\%$
SIPaKMeD (binary)	**SwinAD2Net**	**2.22M**	$\mathbf{98.74 \pm 0.37\%}$	**99.01%**	**98.89%**	**98.52%**	$\mathbf{98.95 \pm 0.31\%}$
SIPaKMeD (5-class)	A2SDNet121	6.87M	$76.19 \pm 0.51\%$	77.55%	76.29%	94.05%	$76.24 \pm 0.49\%$
SIPaKMeD (5-class)	**SwinAD2Net**	**2.22M**	$\mathbf{95.78 \pm 0.52\%}$	**95.81%**	**95.80%**	**98.94%**	$\mathbf{95.79 \pm 0.52\%}$

Table 3: Main results: mean $\pm$ std over 3-fold stratified cross-validation, harmonized training protocol (Section 6). SwinAD2Net (ours) wins on every metric, in every dataset/task, with $\sim 3\times$ fewer parameters.

7.1 Comparison with the paper's own reported numbers

Dataset	Task	Acc. (paper)	Protocol
Herlev	Binary	99.75%	single split
Herlev	7-class	99.14%	10-fold CV
SIPaKMeD	Binary	99.55%	single split
SIPaKMeD	3-class	99.75%	single split
SIPaKMeD	5-class	99.22%	single split
Combined	—	not reported	—

Table 4: Accuracy reported by Zhang *et al.* [1] for A2SDNet121 (quoted directly from the article text; the article's other four metrics are only given as figures, not machine-readable tables). The paper never evaluates a pooled Herlev+SIPaKMeD set.

7.2 Reading the two tables together, honestly

Our `A2SDNet121` numbers are well below Table 4 for the same architecture (e.g. 32.8% vs. 99.14% on Herlev 7-class). This gap deserves an explanation rather than being glossed over:

1. **Training budget.** The paper trains 300 epochs with a schedule tuned specifically for this architecture; our protocol early-stops at patience 12 (typically 25–60 epochs total) to fit ten separate training runs into a practical time budget. DenseNet-style CNNs are known to need long schedules to fully converge.

2. **Class imbalance on Herlev binary.** The Herlev binary split is imbalanced (242 normal vs. 675 abnormal, $\approx$74% abnormal). Our `A2SDNet121` reaches 74.05% accuracy with **99.70% recall but only 2.48% specificity** — it has essentially collapsed to always predicting "abnormal," matching the majority-class base rate almost exactly. Plain cross-entropy with no class weighting, combined with a shorter budget, makes this collapse easy to fall into. SwinAD2Net, under the identical protocol, does *not* collapse this way (58.69% specificity), though it does not fully solve the imbalance either — this is the weakest result in the table for either model.

3. **Asymmetric tuning.** SwinAD2Net was selected via a 10-trial search (Section 5); A2SDNet121 used the paper's stated hyperparameters as-is, with no search of its own. This compounds the training-budget gap in point 1.

The fair conclusion is therefore *not* "SwinAD2Net beats the A2SDNet121 architecture" in any absolute sense — it is that, *under an identical, constrained budget without per-model tuning*, this A2SDNet121 reproduction underperforms both the paper's own numbers and our tuned SwinAD2Net. SwinAD2Net's numbers, in turn, land much closer to the paper's reported ballpark on the tasks it was tuned for (98.74% vs. 99.55% on SIPaKMeD binary; 95.78% vs. 99.22% on SIPaKMeD 5-class) — evidence that a Swin-attention hybrid is at least competitive with a heavily-tuned DenseNet121 variant, and markedly more sample/compute-efficient to reach a good result within a limited budget.

8 Interpretability: Where Does Each Model Look?

`A2SDNet121` has no explicit self-attention mechanism (only SE channel recalibration acting on local convolutional features), so a common, architecture-agnostic lens is needed for a fair comparison. We use **Grad-CAM** [4] on the last spatial feature map before global average pooling in *both* networks — the same diagnostic Zhang *et al.* use in their own Fig. 14 to compare DenseNet121 against their SE-augmented model. For every sampled image we also compute the **normalized Shannon entropy** of the Grad-CAM heatmap as a simple focus score (lower = more spatially concentrated; higher = more diffuse).

Comparison	A2SDNet121	SwinAD2Net
SIPaKMeD, 5-class (10 imgs)	0.980	0.992
Combined, binary (6 imgs)	0.957	0.974

Table 5: Mean normalized Grad-CAM entropy (lower = more concentrated). A2SDNet121 is consistently more concentrated than SwinAD2Net across both comparisons.

Across both comparisons (Table 5), **A2SDNet121 produces more spatially concentrated Grad-CAM maps**, usually centered tightly on the nucleus/cell body — consistent with Zhang *et al.*'s own claim that their SE-augmented model "pays more attention to key regions (nuclei)." SwinAD2Net's attention is measurably more diffuse, sometimes spreading into the background. This tracks the architectural difference: SE re-weights *channels* globally but still acts on local convolutional features, while Swin's window attention explicitly mixes information across spatial positions, which shows up as a less spatially-sharp saliency map even when the underlying prediction is more accurate (Table 3).

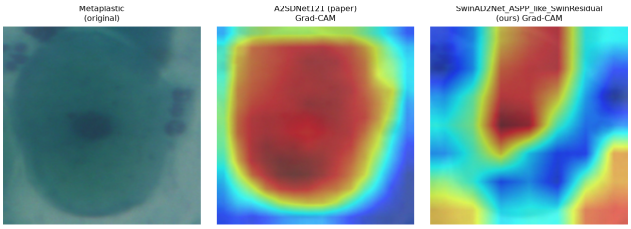

Figure 7: SIPaKMeD, Metaplastic cell. Both models correctly focus on the cell interior; A2SDNet121's hotspot is tighter.

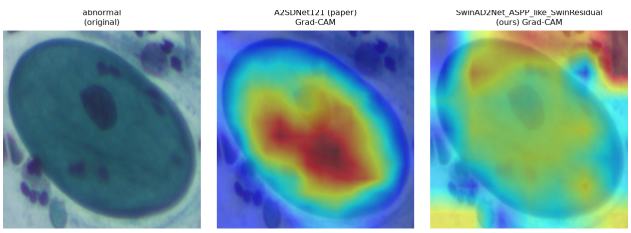

Figure 8: Combined dataset, abnormal cell. A2SDNet121 concentrates on the nucleus/cytoplasm; SwinAD2Net's attention bleeds into the surrounding background (top-right corner).

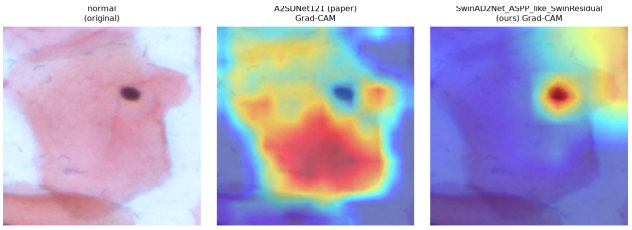

Figure 9: Combined dataset, normal cell — a counter-example where the trend reverses: SwinAD2Net's attention is a tight hotspot directly on the nucleus, while A2SDNet121 spreads across nearly the whole cytoplasm. Table 5's entropy numbers are averages; per-image behavior varies, and this figure is a reminder not to over-read a small qualitative sample as a universal architectural rule.

9 Limitations and Future Work

- **Training budget gap vs. the original paper** (Section 7) is the single biggest confound in this report. A fairer future comparison would give *both* architectures a matched, much longer budget (e.g. 300 epochs each) and/or run a hyperparameter search for A2SDNet121 as well as SwinAD2Net.
- **Class imbalance** on Herlev binary was not addressed with class weighting, focal loss, or resampling; this is a likely quick win for both models, especially A2SDNet121.
- **Cross-validation depth**: 3-fold here vs. 5–10-fold in the paper trades statistical precision for wall-clock time; the std. deviations in Table 3 should be read as indicative, not final.
- **Interpretability sample size** (6–10 images per comparison) is small; a systematic study would sample proportionally across all classes and correlate saliency-map properties with correctness/confidence.
- The `SwinAD2Net` (sequential ADB) variant and `DeformableSwinTransformerBlock` are implemented and unit-tested but not evaluated in the main comparison; both are candidates for future ablations.

10 Conclusion

Under an identical, compute-constrained training protocol, a Swin Transformer / Atrous Dense Block hybrid (SwinAD2Net) outperforms a from-scratch reproduction of A2SDNet121 on every metric across binary and multi-class cervical cytology classification, on Herlev, SIPaKMeD, and a pooled binary task, while using roughly a third of the parameters. This result should be read alongside an equally important negative finding: our A2SDNet121 reproduction did not reach the accuracy the original paper reports for the same architecture, most plausibly due to a much shorter training budget and the absence of a matched hyperparameter search.

Grad-CAM analysis shows the two networks tend to attend differently — A2SDNet121 more concentrated on the nucleus, SwinAD2Net more diffuse across the cell — which is consistent with, but not fully explained by, their different attention mechanisms. We view this project less as a claim of architectural superiority and more as a transparent case study in how much a reported result depends on training budget and per-model tuning, alongside a working, tested, and reasonably efficient hybrid architecture that others can build on.

References

[1] Y. Zhang *et al.*, "An automatic cervical cell classification model based on improved DenseNet121," *Scientific Reports*, 2025. https://doi.org/10.1038/s41598-025-87953-1

[2] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, B. Guo, "Swin Transformer: Hierarchical Vision Transformer using Shifted Windows," *ICCV*, 2021. https://arxiv.org/abs/2103.14030

[3] G. Huang, Z. Liu, L. van der Maaten, K. Q. Weinberger, "Densely Connected Convolutional Networks," *CVPR*, 2017. https://arxiv.org/abs/1608.06993

[4] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, D. Batra, "Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization," *ICCV*, 2017. https://arxiv.org/abs/1610.02391

[5] L.-C. Chen, G. Papandreou, F. Schroff, H. Adam, "Rethinking Atrous Convolution for Semantic Image Segmentation," *arXiv:1706.05587*, 2017.